What happens in one message-passing layer?

Source tile j → destination tile i | The same learned functions are shared across edges and nodes

1 Build each edge message

$m(j \to i) = M_k([z_j, z_i, a_{ji}])$
Use source and destination features.
Edge attributes a are optional.

2 Aggregate at destination

$\bar{m}_i$ = mean of incoming messages
Average over the tile's neighbors.
Degree can vary between tiles.

3 Update the destination

$z_i \leftarrow z_i + U_k([z_i, \bar{m}_i])$
Residual update retains local features.
No incoming edges:
node stays unchanged.

4 Repeat and condition

Repeat for K graph layers.
$c'_i = c_i + P(z_i^K - z_i^0)$
Use c'_i in the U-Net bottleneck.

Reading the symbols

z_i: encoded features for tile i
M_k: learned message MLP
U_k: learned update MLP
P: learned linear projection
[,]: concatenation; k: graph layer

Two layers = two spatial hops

Layer 1 uses adjacent-tile features.
Layer 2 can carry features from two hops away.
These are spatial exchanges, not forecast steps.
All nodes still share the same origin and lead.

How we test its value

Same U-Net and connected context batches:
- U-Net without graph messages
- Local capacity control with self edges
- Graph with actual adjacency edges

Match data, losses and evaluation origins.

Messages have no water-volume interpretation; physical conservation requires a separate formulation.

Implemented local graph prototype; corrected-data forecasting benefit remains unproven.